

Planificarea proiectului

Regulament de notare a proiectului

În acest document vom avea împărțirea în etape a proiectului cu detalii despre taskuri și penalizări.

Deadline final (pentru trimiterea proiectului si referate): duminica din saptamana 13

Bonusuri și penalizări

În săptămâna cu punctaj normal poate fi în continuare cu bonus în următoarele situații:

1. studentul a prezentat în săptămâna cu bonus dar avea greșeli sau taskuri incomplete și a decis să îndrepte situația
2. am reprogramat pe cineva din motive justificate: probleme tehnice (unuia dintre noi nu ne-a mers calculatorul sau netul etc), probleme administrative (nu a putut prezenta având probleme cu căminul, cu secretariatul etc), probleme de sănătate, e om serios și ar fi terminat complet tema dar s-a aglomerat cu alte materii etc.
3. o amânare oficială, de comun acord cu toți studenții, fiind necesare explicații suplimentare la curs/laborator.

Atentie! Perioada de punctaj normal nu va avea bonus aplicat dacă studentul se prezintă pentru prima oară cu etapa proiectului rezolvată și fără o amânare justificată.

Bonusul e în valoare de ~~40%~~ 15%

Penalizările sunt de 10%.

Alte precizări

- **Conteaza ora la care ati trimis etapa si nu momentul prezentarii.** Puteti trimite etapa si daca e undeva pe la 90% gata sau cu mici buguri si tot consider bonusul cat timp imi spuneti situatia, si chiar puteti remedia problema pana la prezentare. (evident nu se considera bonusul pentru o etapa care e abia inceputa sau cu foarte multe lucruri gresite). De asemenea consider bonusul si daca ati intarziat cateva minute (nu cateva zile!). Deci incercarea mea e sa nu vedeti deadline-ul ca fiind ceva stresant ci dimpotriva sa aveti curaj sa trimiteti ce ati lucrat si sa puneti interbari cu privire la ce nu a mers.
- **O etapa poate fi prezentata oricand in cadrul semestrului** inasa e bine sa prezentati cat mai curand cand aveti codul proaspat in minte. De asemenea, daca aveti greseli ori lucruri lipsa la o etapa, care ar duce la depunctari, puteti sa recuperati punctele in etapa urmatoare, daca ati modificat/completat acele lucruri in cadrul proiectului.
- **În cazul depunctărilor rezultate din greșeli sau lipsa unor taskuri, se poate recupera punctajul dacă remediați acele probleme până la prezentările următoare.** Punctele nu sunt pierdute pentru totdeauna, puteți aduce completări în săptămâna 14 pentru etapa 1 fără a afecta bonusul sau penalizarea etapei deja scrise în tabel.
- **Va rog sa trimiteti etapele pe teams** (fisierul html), nu pe mail sau pe site. Cei care au pus proiectul pe github doar imi trimit linkul sau daca l-au trimis deja, **anunta pe teams ca au facut modificarea (mesajul cu anușul trebuie trimis înainte de deadline, ca să pot verifica).**
- Etapele de la 1 incolo trebuie prezentate pentru a lua puncte pe ele.
- **Bonusurile din cadrul unei etape pot fi rezolvate și mai târziu fără penalizări** (adică puteți, de exemplu să faceți bonusul dupa 3 săptămâni de la deadline) însă bonusului i se va aplica penalizarea sau bonusul de timp al etapei. De exemplu, etapa a fost făcută cu bonus de timp, și după 3 săptămâni de la deadline

trimiteți și un task bonus, i se va aplica și lui bonusul de timp al etapei (iar dacă există penalizare de timp i se va aplica penalizarea). Motivul: încurajarea studentului de a face taskurile obligatorii la timp..

Linkuri utile pentru proiect

<https://pixabay.com/> (imagini gratuite)

<https://www.videvo.net/> (videoclipuri gratuite - cele marcate cu "free")

Etapale proiectului

Etapa	Perioadă bonus	Perioadă punctaj normal	De când încep penalizările săptămânale
Etapa 0 (0.3)		02.03. 2026	
Taskuri etapa 0 (0.35) (0.025) Alegerea temei (titlul site-ului) - Descrierea succintă a temei (google docs, 1/2-1 pagina). Va contine următoarele informații: (0.025) Impartirea informatiilor, serviciilor etc. pe categorii si subcategorii. (0.025) Identificarea efectiva a paginilor (încercați să fie doar 4-5) și partial a legaturilor dintre ele (ce informatii vor avea pagini separate si care informații vor fi subsecțiuni în aceeași pagină) (0.025) Stabilirea cuvintelor/sintagmelor cheie (o lista cu cuvintele cheie ale site-ului cât și cate o lista pentru fiecare pagină în parte) (0.2-0.25) Căutarea unor site-uri similare (4-5 site-uri) ca tema pentru a observa modul de organizare a informației. Veti observa pentru fiecare site cum au impartit informatiile si cum au facut designul. Veti nota pentru fiecare site ideile demne de implementat, dar și neajunsurile acelor site-uri (pentru fiecare site minim 2 lucruri pro si 2 contra). Veți pune linkuri către ele în fișier.			
Etapa 1 (punctaj recomandat 1.1)	09.03. 2026	16.03. 2026	
Taskuri etapa 1 Creați prima pagină a site-ului (doar prima pagină; fără stilizare încă, fiindcă veți primi taskuri legate de acest aspect). Puteți pune în această pagină text care va fi mutat în alte pagini, mai târziu, dar nu faceți încă mai multe pagini fiindcă le vom genera prin Node! La prezentare vă rog să aveți pentru fiecare task notată linia din program la care l-ați rezolvat ca să nu dureze prezentarea mai mult de 3-4 minute. Tagurile și atributele cerute mai jos trebuie să fie relevante în raport cu conținutul și tema de proiect pentru a fi punctate.			
Punctaj total pentru cerințe: (0.025) Creați un folder al proiectului care va cuprinde toate fisierele necesare site-ului vostru. Creați în el un fisier numit index.html. Deschideți acest fișier cu un editor de text care marchează sintaxa. Adăugați în fișier doctype și setați limba documentului în tagul html (0.05) Adaugați un title corespunzător conținutului textului. Folosiți 4 taguri meta relevante pentru a specifica: charset-ul, autorul, cuvintele cheie, descrierea. Punctajul se va da în funcție de cât de precise și relevante sunt conținuturile tagurilor meta. (0.025) Textul din pagină trebuie să conțină toate cuvintele cheie decise pentru pagina curentă (și enumerate în tagul meta). Puteți găsi mai multe sintagme cheie pe care le puteți folosi, cu https://www.wordtracker.com/ sau https://app.neilpatel.com/en/ubersuggest/keyword_ideas. Acestea trebuie să apară de mai multe ori în pagină, în taguri relevante. (0.025) Creați un folder (de exemplu numit "resurse") care va conține toate fisierele folosite de site, dar care nu sunt pagini html (de exemplu imagini, fisiere de stilizare etc). In el creati un folder numit ico. Adaugați un favicon relevant pentru temă. Folosiți https://realfavicongenerator.net pentru a genera toate dimensiunile necesare de favicon și codul compatibil pentru diversele browsere și sisteme de operare. Pentru favicon transparent, trebuie sa setati si o culoare a tile-ului (de background), care trebuie specificata și în tagul meta: <meta name="msapplication-TileColor" content="...culoarea aleasa de voi...">			

5. (0.025) Împărțiți body-ul în header, main, footer.
6. (0.05) Folosiți minim un tag dintre: section, article, aside. Trebuie să existe măcar un caz de taguri de secționare imbricate (secțiuni în secțiuni). Puneți headingul cu nivelul corespunzător nivelului imbricării. Atenție, nu folosim headinguri decât ca titluri pentru tagurile de secționare. **Observație:** nivelul headingului trebuie să corespundă nivelului de imbricare a secțiunii (de exemplu un tag de secționare aflat direct în body are titlul scris cu h2, dar un tag de secționare aflat într-un tag de secționare care la rândul lui se află în body, va avea titlul scris cu h3)
7. (0.025) Minim un heading din pagină va fi acompaniat de un subtitlu, folosind tag-ul <hgroup>.
8. (0.05) În **header** faceți un sistem de navigare ca în curs (nav cu listă neordonată de linkuri), cu opțiuni principale (care vor reprezenta paginile site-ului) și secundare (pentru opțiunea "Acasă", adică pagina principală, subopțiunile vor cuprinde linkuri către secțiunile paginii, care vor avea id-uri relevante). Subopțiunile vor fi puse într-o listă **imbricată** în lista de opțiuni principale. Folosiți în header tagul h1 pentru titlul site-ului.
9. (0.025) În cadrul secțiunilor folosiți minim 2 taguri dintre următoarele taguri de grupare: p, blockquote, dl
10. (0.05) Veți crea o secțiune de evenimente, cu date și ore (folosind tagul <time> și atributul datetime cu informații de dată și oră). Evenimentele vor fi enumerate într-o listă ordonată () sau neordonată (). Evenimentele vor avea un nume pus în tagul , urmat de o descriere. De exemplu: 5 ianuarie ora 12, **Prajituri cadou** - oferim tuturor vizitatorilor câte o prăjitură gratis în funcție de stocul disponibil.
11. (0.05) Adăugați în pagină o imagine cu descriere, folosind figure și figcaption. Imaginea va avea și o descriere mai scurtă în atributul title (care se vede când venim cu cursorul pe ea). Pe ecran mic (mobil) trebuie să se încarce o variantă mai redusă în dimensiune (bytes) a imaginii, pe tabletă o variantă medie, iar pe ecran mare varianta cea mai mare a imaginii (folosiți tagul picture). Folosiți un editor grafic pentru cropping și redimensionare pentru a obține cele 3 variante de imagini.
12. (0.075) În cadrul textului îndepliniți **3** dintre cerințele de mai jos, **la alegere**:
 - a. marcați minim 3 cuvintele și sintagme cheie cu ajutorul tagului b
 - b. Marcați minim 2 cazuri de text idiomatic (termeni științifici, în altă limbă, termeni tehnici, de jargon, etc) cu tagul i. Trebuie să fie termeni relevanți pentru site
 - c. Realizați un text care anunță ceva urgent, important și marcați-l cu tagul strong
 - d. În cadrul textului folosiți un cuvânt în mod ironic, sau considerați că puneți accentul (la citire) pe un cuvânt din text. Marcați cuvântul accentuat cu em
 - e. marcați textul șters (corectat sau care nu mai e relevant) cu tagul s și textul inserat în loc cu tagul ins
 - f. marcați o abreviere (relevantă pentru site) cu tagul abbr și cu atributul title specificați sintagma abreviată
 - g. Alegeți un termen relevant temei, dar mai puțin cunoscut, dintre cei care apar pe site. Marcați acel termen definit cu dfn, și scrieți definiția pentru el
 - h. marcați un citat relevant pentru conținutul site-ului cu tagul q
 - i. marcați titlul unei lucrări citate cu tagul cite
13. (0.1=5*0.02) Creați următoarele linkuri (folosind tagul <a>), cu mențiunea că trebuie să fie relevante în pagină:
 - a. un link către o resursă externă (de exemplu un alt site cu temă relevantă pentru conținut). Acest link nu se va plasa în meniu. Linkul se va deschide într-un tab nou.
 - b. un alt link către un element dintr-o resursă externă (referențiat prin id). Exemplu: <http://ceva.com#referinta>. Textul tagului <a> va fi chiar adresa indicată de link. Adresa trebuie să fie mai lungă. Dacă linkul nu are loc pe rândul său, trebuie să se separe în bucăți trecând pe rând nou (vedeți exemplul din curs de la <wbr>)
 - c. un link în footer care duce către începutul paginii.
 - d. un link care conține o imagine. La click pe imagine, se va deschide aceasta în tabul curent, într-o versiune mai mare.
 - e. un link de tip download, care va seta numele fișierului de descărcat la altul decât cel inițial (de exemplu, se numea poza.png dar se descarcă automat cu numele poza_descarcata.png). Pentru ca descărcarea să funcționeze, pagina html trebuie să funcționeze pe un server. Descrieți extensia Live Server (pentru VSCode) și apăsați pe "Go Live" care apare în dreapta-jos (poate necesita restart după instalare).
14. (0.05) Creați un iframe în pagină care conține un videoclip de youtube embedded (vedeți: https://developers.google.com/youtube/player_parameters). Imediat deasupra iframe-ului într-un tag div veți crea minim două linkuri (tagul <a>) către alte videoclipuri de youtube și un al treilea link către videoclipul de youtube deja încărcat în iframe. Toate aceste linkuri se vor deschide în iframe (vedeți exemplul de la curs).
15. (0.1) Creați un tabel cu sens în site (de exemplu: cu oferte, cu programul de lucru, cu detalii despre evenimentele importante care urmează, cu date despre cei mai buni angajați, cu date despre niste proiecte în care a fost implicată firma pentru care faceți site-ul, cu date despre cele mai vândute produse etc.). Tabelul trebuie să fie împărțit în thead (antet), tbody, tfoot (fiecare conținând date relevante). În antet vor fi celule cu numele coloanelor (folosiți th). Tabelul trebuie să aibă minim 5 rânduri și 4 coloane. Minim una dintre celule va avea rowspan sau colspan setat la valoare diferită de 1. Tabelul va avea și descriere (caption).
16. (0.025) Creați în pagină mai multe zone de details și summary. Pot fi întrebări frecvente, pot fi niște oferte pentru care afișăm titlul și utilizatorul le deschide pe cele care îl interesează, pot fi secțiuni explicative etc.
17. (0.05) Folosiți în site tagul meter (de cel puțin două ori în pagină) setând o valoare existentă, minimul, maximul, de la ce punct se consideră valoare mică, de la ce punct se consideră valoare mare și care e valoarea optimă. Un tag meter va avea o valoare mică (până în limita *low*) iar celălalt valoare mare (peste valoarea atributului *high*). Exemple de folosire: număr de participanți confirmați la evenimente, review-uri date de ziare, utilizatori etc cu o notă din partea lor, numărul de clienți într-un anumit interval orar în raport cu capacitatea magazinului etc.)

18. (0.1 = 4 * 0.025) În footer se vor adăuga cu ajutorul tagului address informații de contact:
 - a. telefon fictiv, marcat cu tagul <a> și URI Scheme-ul corespunzător
 - b. adresă fictivă care la click deschide o locație pe Google Maps (locația în mod normal ar corespunde cu adresa dar voi pune drept locație în maps, Facultatea de Matematica și Informatică). Adresa fictivă va fi separată cu
 pe două sau mai multe rânduri.
 - c. e-mail fictiv, marcat cu tagul <a> și URI Scheme-ul corespunzător în href
 - d. Link care deschide o aplicație de comunicare precum [whatsapp](#) pentru chat
19. (0.05) În footer se va adăuga informație de copyright, folosind tagul small, simbolul specific de copyright cu codul html necesar (forma &cod;) și data creării paginii scrisă în limba română și pusă în tagul time cu atributul datetime corespunzător (doar cu informații despre dată, nu și oră).
20. (0.15) **Cerință obligatorie!** Pagina trebuie să fie validă din punct de vedere sintactic. Deci verificați cu [validatorul html](#). Validatorul va fi pregătit într-un tab, la prezentare, și pagina se va valida pe loc. **Doar după ce s-a demonstrat că pagina e validă se începe prezentarea.**

Se poate da bonus pentru o temă bine făcută sau pentru folosirea mai multor taguri decât minimul specificat (dar studentul trebuie să anunțe la prezentare că le-a folosit)

Evitați pentru moment să adăugați alte taguri fiindcă vor apărea în taskurile următoare.

Bonusuri:

1. **Bonus (0.05-0.2 în funcție de complexitate)** Folosirea unei formule scrise în MathML - formula trebuie să aibă sens în contextul site-ului.
2. **Bonus (0.05)** Afișarea unui pdf în pagină cu ajutorul tagului <embed> sau <object>
3. **Bonus (0.2-0.3 în funcție de complexitate)** Crearea unei hărți de imagini, folosind tagurile <map>, <area>
4. **Bonus (0.05)** Adăugați pentru adresa facultății (de la un subpunct anterior) și un iframe cu locația marcată pe google maps.
5. **Bonus (0.05)** Crearea unui iframe în care se afișează pe rând (în mod automat) videoclipurile dintr-un [playlist](#) de pe youtube (playlist-ul va avea minim 2 videoclipuri), cu butoanele de controlare a videoclipurilor afișate. Playlist-ul se va relua automat de la început după redarea ultimului videoclip. Cerința se va rezolva doar din parametri în cadrul linkului youtube. Documentație: https://developers.google.com/youtube/player_parameters

Etapa 2 (punctaj recomandat 0.7)

18.03. 2026

23.03.2026

Taskuri etapa 2 (punctaj recomandat: 0.4)

Atentie - unele cerințe au enunț diferit pentru fiecare student (și sunt marcate printr-un link). Trebuie să vă înregistrați pe site pentru a le vedea.

Dacă stilizarea dintr-o cerință nu vă place, puteți să imi cereți o altă variantă (imi scrieți pe chat). Culoarele din imaginile și videoclipurile date ca exemplu nu trebuie respectate (folosiți culoarele din schema cromatică aleasă de voi).

(0.05) Se va face un repository local și se va pune proiectul pe Github.

(0.05) Task schema cromatică: ([cerință individuală](#))

(0.15) Task layout: ([cerință individuală](#))

(0.25) Task design rudimentar:

- a. Folosind variabile CSS, să se adauge o spațiere în stânga și dreapta paginii, comprimând un pic conținutul, dar aerisind pagina. Spațierea trebuie să fie identică în cele două direcții. Spațierea va fi mai mică pe ecran mediu și de valoare minimă pe ecran mic.
- b. Folosiți gap pentru a realiza o spațiere între celulele gridului paginii. Spațierea trebuie să descrească pe ecran mediu și mic.
- c. Izolați vizual zonele paginii: header, footer, și zonele gridului folosind minim 3 dintre următoarele efecte CSS: background de culoare (aleasă din schema cromatică) diferită pe zone diferite, border, colțuri rotunjite ale box-ului, box-shadow
- d. Folosiți padding pentru a distanța textul din zone de granițele zonelor. Padding-ul ar trebui să fie egal pentru toate zonele de text. Folosiți variabile CSS în cazul în care nu puteți asigura acest lucru prin simplii selectori.
- e. Elementele media (imagini, videoclipuri etc.) care vin cu o lățime presetată vor primi lățimea în procente, setând, totuși, o lățime maximă și minimă pentru ele, pentru a nu avea efecte vizuale nedorite. Aceste valori pot să difere în funcție de dimensiunea ecranului

(0.05) Task iconuri și font extern. Folosiți în prima pagină a site-ului un [font extern prin Google API](#). Folosiți în pagină, într-un loc relevant un icon static și unul animat (**diferite de iconurile cerute în eventuale alte taskuri**) din colecția [Font Awesome](#) (fie folosind kit - recomandat - fie folosind linkul CDN ca în exemplu: <https://replit.com/@IrinaCiocan/curs4-exemple#fontawesome.html>)

(0.05) Stilizare tabel: ([cerință individuală](#))

Indicații de rezolvare:

1. Pentru taskul cu valori alternate folosiți pseudoclaselor :nth-child(odd), :nth-child(even) cu elementele cerute (td pentru celule(coloane) și tr pentru rânduri)
2. Proprietățile de care aveți nevoie sunt în special la [CSS Styling Tables \(w3schools.com\)](https://www.w3schools.com/css/css3-pseudo.php)
3. Bara de scroll se poate realiza cu ajutorul proprietății overflow.

(0.05) Task stilizare-taburi: ([cerință individuală](#))

Indicații de rezolvare:

1. Creați un container în care puneți linkurile care se deschid în iframe (cel care are clasa "taburi-iframe" în exemplul de mai jos). De asemenea grupați containerul linkuri și iframe-ul într-un alt container (cel cu clasa container-iframe din exemplul de mai jos)

```
<div class="container-iframe">
  <div class="taburi-iframe">
    <a href="https://www.youtube.com/embed/qtlhdlfojmc" target="ifr-video">Reteta tort vanilie</a>
    <a href="https://www.youtube.com/embed/dsJtgmAhFF4" target="ifr-video" >Reteta tort ciocolata</a>
    <a href="https://www.youtube.com/embed/Wtxbt-CpA2s" target="ifr-video" >Reteta tort căpșuni</a><br/>
  </div>
  <iframe name="ifr-video" width="560" height="315" src="https://www.youtube.com/embed/qtlhdlfojmc"
    frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
    allowfullscreen></iframe>
</div>
```

2. Stilizați tagurile <a> sub forma de butoane (width și height setat, background, border)
3. Aplicați display:flex pe containerul care trebuie să conțină coloane (fie container-iframe, fie taburi-iframe în funcție de cerință)

(0.05) Link top: ([cerință individuală](#))

Indicație de rezolvare: Porniți de la exemplul: [exemplu_layout-1 - Replit](#) - cautați în html elementul cu id-ul "link-top", și fiul lui care e un div cu id-ul "triunghi". Preluati codul css pentru ele și îl modificați pentru a obține rezolvarea la acest task.

-
-

Bonusuri:

- (0.05 - 0.15 în funcție de complexitate) Resetarea css-ului cu redefinirea spațiilor, dimensiunilor, culorilor, stilurilor bol și italic, eventual a bulleturilor și indicilor de listă, a stilului tabelelor). În afară de body și html care vor primi dimensiuni în unități fixe, toate celelalte elemente vor folosi unități relative. Se vor folosi variabile pentru valori care se repetă și depind logic unele de altele. Puteti folosi de exemplu: <https://meyerweb.com/eric/tools/css/reset/> Resetarea css-ului se va face într-un fișier separat
- (0.05-0.1 în funcție de complexitate) Stilizarea unei formule scrise în MathML: părți diferite din formulă trebuie să aibă stil diferit, de exemplu schimbarea culorii, a dimensiunii de font, italicizarea sau îngroșarea caracterelor etc.)

Etapă 3 (punctaj recomandat 0.4)

30.03.2026

02.04.2026

Taskuri etapă 3

Atentie - unele cerințe au enunț diferit pentru fiecare student (și sunt marcate printr-un link). Trebuie să vă înregistrați pe site pentru a le vedea.

(0.25) Task meniu: ([cerință individuală](#))

(0.15) Stil printare ([cerință individuală](#))

Bonusuri:

- (0.05) Icon-ul meniului "hamburger" să fie creat cu 3 divuri/span (eventual puse într-un div container în locul unei imagini), cele 3 divuri vor primi background, width și height pentru a simula dreptunghiuri și vor fi poziționate absolut în interiorul containerului.
- (0.05) Când se trece pe ecran mic și apare iconul pentru meniu, apariția să fie făcută printr-o animație asupra divurilor care să implice schimbarea tuturor următoarelor proprietăți: culoarea celor 3 bare, o transformare geometrică, opacitate. Puteți schimba și alte proprietăți dacă doriți. Animația trebuie să aibă minim 3 cadre cheie.
- (0.05) Pentru bonusul anterior, fiecare bară din hamburger-menu să aibă asociată o animație, însă animațiile să înceapă succesiv cu o diferență de t milisecunde (de exemplu t=300). Delayurile diferite în cadrul animației se vor genera cu o instrucțiune for scrisă în sass.

Etapa 4	06.04.2026	20.04.2026	
Taskuri etapa 4 (punctaj recomandat 1.15p) (1p) Trecerea site-ului pe node și crearea de fișiere EJS conform cerințelor: - În folderul proiectului dați comanda "npm init" și setați numele, autorul, descrierea și cuvintele cheie pentru proiectul vostru. Instalați express și ejs. - Se va crea în rădăcina proiectului un fișier index.js. În el se va crea un obiect server express care va asculta pe portul 8080. (sau alt port dacă aveți deja folosit 8080) - Să se afișeze calea folderului în care se găsește fișierul index.js (__dirname), calea fișierului (__filename) și folderul curent de lucru (process.cwd()). Sunt __dirname și process.cwd() același lucru întotdeauna? - Se va folosi EJS pentru generarea (randarea) paginilor. Se va face un folder numit views în rădăcina proiectului. În el veți face un folder numit pagini (care conține paginile întregi) și altul numit fragmente (care conține părți de pagini (bucățele de cod html) ce pot fi refolosite pe mai multe pagini). Instalați în Visual Studio Code extensia EJS language support. - Din index (care va fi redenumit index.ejs) se vor decupa headerul și footerul și se vor pune în ejs-uri separate. De asemenea se va decupa partea de head care conține codul care nu se schimbă în funcție de pagină (de exemplu, tagul meta cu encodingul sau autorul, includerea faviconului, fișierelor css generale (nu specifice paginii) a scripturilor generale etc). Se va folosi funcția include() în fișierele ejs, pentru a include toate aceste fragmente în pagini - Se va realiza (dacă nu l-ați făcut deja) un folder special cu toate resursele site-ului (în stilul exemplului de la curs în care am pus toate fișierele statice, precum imagini, fișiere de stil, videoclipuri etc în folderul "resurse"). Numele folderului îl decideți voi, însă va trebui să fie structurat, de asemenea, în subfoldere în funcție de tipul și modul de utilizarea al fișierelor. Se va defini în program acest folder ca fiind static - Se vor schimba căile fișierelor-resursă folosite în pagini, astfel încât să nu mai fie relative ci stil cerere către server (de exemplu, /resurse/stiluri/ceva.css în loc de, de exemplu, ../resurse/stiluri/ceva.css) - Prima pagină (index) trebuie să se poată accesa atât cu localhost:8080 cât și cu localhost:8080/index, localhost:8080/home. Realizați acest lucru folosind un vector în apelul app.get() care transmite pagina principală - Veți declara un app.get() general pentru calea "/", care tratează orice cerere de forma /pagina randând fișierul pagina.ejs (unde "pagina" e un nume generic și trebuie să funcționeze pentru orice string). Atenție, acest app.get() trebuie să fie ultimul în lista de app.get()-uri. Dacă pagina cerută nu există, se va randa o pagină specială de eroare 404 (în modul descris mai jos). - Se va da ca argument în funcția render o funcție callback function(eroare, rezultatRandare) care, în cazul în care mesajul erorii începe cu "Failed to lookup view" va afișa pagina pentru eroarea 404 (vezi mai jos). În cazul în care e altă eroare va afișa pagina de eroare generică (vezi mai jos), iar dacă nu sunt erori va trimite către client rezultatul randării. - Pentru randarea erorilor, veți folosi un fișier json, numit erori.json. Acesta va avea următoarele proprietăți: - cale_baza: calea la care se găsesc imaginile corespunzătoare erorilor - eroare_default: va fi un obiect JSON cu proprietățile: titlu (titlul paginii de eroare), text, imagine - info_eroi: un vector de obiecte. fiecare obiect descrie o eroare și are proprietățile: identificator (un cod numeric; pentru erorile http, precum 403, 404 e chiar codul http), status (boolean prin care indicăm dacă trebuie alt cod status decât 200 pentru răspuns), titlu (titlul erorii, pus în heading), text (descrierea erorii), imagine(o imagine descriptiva pentru eroare; se va da calea relativa la cale_baza). Se vor adăuga în proiect în calea specificată în cale_baza imagini pentru erorile definite. - Se va crea un template (eroare.ejs) cu ajutorul căruia să se afișeze erorile. Acesta va avea, preluate din locals, titlul, textul imaginea erorii. - Se va crea o variabilă globală numită obGlobal de tip obiect. Aceasta va avea o proprietate numită obErori, implicit cu valoare null. Se va crea o funcție, numită initErori(), fără parametri, care citește JSON-ul cu erorile și creează un obiect corespunzător lui cu toate datele erorilor (pentru a le avea încărcate în memorie), acest obiect va fi salvat în proprietatea obErori a variabilei obGlobal. Pentru fiecare eroare, se va seta calea absolută în proprietatea imagine (folosind proprietatea cale_baza) - Se va crea o funcție de afișare a erorilor, numită afisareEroare() care va primi un obiect de tip Response, identificatorul, titlul, textul și imaginea erorii. În cazul în care există o eroare cu acel identificator, și titlul, textul și imaginea nu sunt precizate, se preiau datele încărcate din JSON pentru afișarea erorii. Dacă una dintre cele 3 proprietăți ale erorii e dată ca argument în funcție, are prioritate asupra datelor din JSON și se va afișa în pagină (de exemplu dacă titlul e dat ca argument, se afișează argumentul nu titlul citit din JSON). În cazul în care identificatorul nu se specifică, se afișează o pagină de eroare cu datele din eroare_default, însă tot cu posibilitatea de a afișa titlul, textul și imaginea din argumente, dacă sunt precizate. - Veți mai face încă o pagină (cu puțin text sau imagini, ca să aibă conținut), de exemplu o pagină cu descrierea site-ului sau istoricul său, al firmei virtuale pentru care este făcut etc. Această pagină trebuie să poată fi accesată prin meniu (linkul să fie corect și să			

transmită o cerere de tip get). Nu faceti încă pagina de produse, fiindcă pe acelea le preluăm din baza de date. Nici paginile de înregistrare sau login, fiindcă le tratăm separat.

16. În zona din layout de date despre utilizator vom afișa ip-ul utilizatorului (prin program). Deocamdată, site-ul fiind local, veți vedea mereu ip-ul de localhost (adică ::1). Ip-ul real se va vedea când adăugați site-ul pe Internet.
17. La o cerere către o cale din /resurse(de exemplu,localhost:8080/resurse/css/) fără fișier specificat (către folderul care ar conține fișierul) se va returna eroarea 403 Forbidden. Pagina de 403 va avea format similar cu cea de 404, folosind același template (eroare.ejs), dar textul și imaginea schimbate corespunzător, preluate din JSON
18. La cererea oricărui fișier cu extensia *ejs* se va transmite o eroare de tip 400 Bad Request. Pagina de 400 va folosi același template (eroare.ejs), dar textul și imaginea schimbate corespunzător, preluate din JSON
19. Să se adauge un `app.get()` pentru `"/favicon.ico"`. Uneori browserele cer favicon pentru diverse răspunsuri primite de la server (nu neapărat fișiere html, unde putem specifica faviconul prin taguri link). Pentru această cerere vom trimite un favicon cu metoda `sendFile()`.
20. Proiectul nostru va folosi niște foldere în care generează fișiere. Scrieți un vector cu numele folderelor de creat , numit `vect_foldere`, (vectorul va conține numele folderelor: "temp", "logs", "backup", "fișiere_uploadate"). Se va itera prin vector și se va testa dacă folderele există. Dacă un folder nu există, este creat. Peste tot unde aveți concatenare de căi folosiți `path.join()`.

(0.05) Task video ([cerință individuală](#))

(0.05) Task linkuri ([cerință individuală](#))

(0.05) Adăugați în `.gitignore` folderul `node_modules` împreună cu cele create de aplicație (subpunctul 20)

Bonus:

Se va implementa o funcție de verificare a datelor din JSON-ul de erori, care va fi apelata la pornirea serverului. Funcția va afișa mesaje de eroare (cu text clar, detaliat și relevant care să explice problema, pentru a fi remediată) în următoare situații:

- A. (0.025) Nu există fișierul `erori.json` - caz în care, pe lângă afișarea mesajului, aplicația se va și închide (`process.exit()`)
- B. (0.025) Nu există una dintre proprietățile: `info_erori`, `cale_baza`, `eroare_default`
- C. (0.025) Pentru eroarea default lipsește una dintre proprietățile: `titlu`, `text` sau `imagine`.
- D. (0.025) Folderul specificat în `"cale_baza"` nu există în sistemul de fișiere
- E. (0.05) Nu există (în sistemul de fișiere) vreunul dintre fișierele `imagine` care sunt asociate erorilor. Veți modifica `json-ul` de erori astfel încât fiecare eroare să aibă altă imagine.
- F. (0.2) Pentru un obiect din fișier există o proprietate specificată de mai multe ori (de exemplu apare de două ori proprietatea `titlu` (atenție verificarea trebuie făcută pe `string`, nu pe obiectul rezultat din fișier).
- G. (0.15) Există mai multe erori (în vectorul de erori) cu același identificator (în mesajul de eroare se vor preciza toate proprietățile erorilor, în afară de identificator, pentru a le găsi ușor)

Etapă 5

24.04.2026

27.04.2026

Etapă 5 (1.1p)

(0.35) Galeria statica ([cerință individuală](#))

(0.25) **Compilare automată scss.** Se vor realiza următoarele subpuncte:

- a. **Pregătire cadru de lucru.** Se vor defini în obiectul global două proprietăți `folderScss` și `folderCss` care conțin căile din folderul de resurse (depinzând de `__dirname`). Se va adăuga folderul `backup` la lista folderelor create automat de aplicație (așa cum e și folderul `temp`)
- b. **Funcția de compilare a scss-urilor.** Se va face o funcție `compileazaScss(caleScss, caleCss){}` care compilează un fișier `scss` în fișier `css`. Primii 2 parametri reprezintă căile către fișierul `scss` (inputul funcției) și fișierul `css` (outputul funcției). Dacă avem căi absolute se iau fișierele de la cele două căi, iar dacă sunt relative se vor considera relative la `folderScss`, respectiv `folderCss`. compilarea se va face cu ajutorul pachetului `sass`. Dacă numele/calea fișierului `css` lipsește, se va salva în `folderCss` rezultatul compilării folosind numele fișierului `scss`, dar cu extensia `css`
- c. **Salvare în backup.** În cadrul funcției `compileazaScss`, înainte de compilarea automată a `scss-ului` în fișierul `css` asociat, fișierul `css` vechi cu același nume va fi copiat în subcalea `resurse/css` a folderului `backup`. Orice folder din această subcale va fi creat dacă nu există deja. Se va afișa un mesaj de eroare în cazul eșecului copierii.
- d. **Compilare inițială.** La pornirea serverului, toate fișierele `scss` din `folderScss` trebuie să fie compilate în fișierele `css` cu același nume folosind funcția `compileazaScss`. Înainte de suprascrierea fișierului `css`, acesta va fi copiat în folderul `backup` (suprascriind un backup cu același nume - sau dacă vreți să păstrați backup-urile anterioare puteți integra în nume o informație cu privire la timpul creării.

- e. **Compilare pe parcurs.** Se va scrie cod (folosind `fs.watch()`) astfel încât să se urmărească modificările din folderul de fișiere scss. La modificarea/crearea unui fișier acesta va fi compilat automat în css. Fișierul css va avea același nume cu fișierul scss, având doar extensia scss schimbată în css. Înainte de suprascrierea fișierului css, acesta va fi copiat în folderul backup (suprascriind un backup cu același nume - sau dacă vreți să păstrați backup-urile anterioare puteți integra în nume o informație cu privire la timpul creării).

(0.25) **Customizare Bootstrap** cu schema cromatică aleasă de voi și cu dimensiuni de ecran diferite pentru ecrane medii și mari. Veți face un fișier numit `custom.scss` în care folosiți fișierul scss al bootstrap, schimbând valori pentru:

- culorile de background pentru minim 2 teme la alegere pe care aplicați tema cromatică schimbată (customizată de voi)
- culori de font (adică ale literelor. **Precizare:** nu neaparat pentru toata pagina, orice culoare de litere - de exemplu dintr-un buton)
- dimensiuni de ecran diferite pentru ecrane medii și mari
- dimensiunea razelor de border
- dimensiunea literelor headingurilor (h1,h2 etc)
- familia de font implicită
- încă una sau mai multe variabile alese de voi

Corectare Bootstrap. Atenție, este posibil ca integrarea bootstrap să afecteze aspectul site-ului deoarece pentru anumite elemente din pagină v-ați bazat pe aspectul implicit al acestora, prin urmare va fi nevoie să definiți reguli css astfel încât site-ul să revină la forma inițială. CSS-ul pentru bootstrap trebuie pus primul pentru a asigura suprascrierea proprietăților pentru selectorii-tag.

Veți folosi unul sau mai multe elemente de bootstrap care să ilustreze schimbările, dintre cele de mai jos:

Customizarea se va face ca la laborator folosind sass, **iar compilarea se va face automat la repornirea serverului folosind funcția compileazaScss!**

(0.25) **(Efecte CSS)**

Observație: având în vedere că depășirea punctajului pentru lista de efecte de mai jos se transformă în bonus adunat la proiect, aceasta lista poate fi extinsă pe tot parcursul semestrului (deci inclusiv după deadline-ul etapei 4), adăugând noi idei pentru bonusuri, deoarece bonusurile pot fi prezentate și mai târziu. **Atenție! Anumite efecte au cerință individuală asociată (care trebuie respectată, pentru a fi punctate) - enunțurile acestora au identificatorul prefixul "efect_css".** Cele care nu au textul "cerința individuală" au enunțul întreg în acest fișier.

Efecte css propuse (se vor **alege** efecte css de implementat ca să **însușeze 0.25** - nu e obligatoriu să le faceți pe toate din listă)

- (0.05) **Duotone** ([cerință individuală](#))
- (0.15) **Reflexie** ([cerință individuală](#))
- (0.025) **Scrierea textului pe coloane**, folosind proprietatea `column-count`. Se va alege o secțiune cu mai mult text pe care se va aplica proprietatea `column-count`. Pe ecran mic și mediu se va afișa o singură coloană. Între coloane se va afișa și o linie despărțitoare (`column-rule`)
- (0.025) **Schimbarea afișării implicite a textului selectat**, folosind pseudo-clasa `::selection` - schimbați minim 2 proprietăți ale textului selectat, folosind variabilele definite pentru schema cromatică.
- (0.05) text care se plimba orizontal sau vertical printr-o animație (keyframes) recurentă (după ce se termină mesajul, reapare). Elementul cu textul trebuie să fie responsive (să nu apară scrollbar orizontal pe pagină din cauza lui)
- (0.05) **background fix la scroll** într-una din pagini, folosind `background-attachment`. Imaginea de background se va schimba (printr-o animație) după t secunde (t e ales de voi).
- (0.05) **Afișarea unui tabel astfel încât să fie responsive**, conform [exemplului dat](#). Tabelul ales trebuie să aibă minim 4 coloane și să nu conțină celule cu `rowspan/colspan`.
- (0.025) **Afișarea unui tabel transpus** pe o dimensiune de ecran (media query) conform [exemplului](#). Pe restul dimensiunilor de ecran se va afișa la fel.
- (0.1) **Stilizare hr** ([cerință individuală](#))
- (0.05) **videoclip care se comporta ca un background**, conform exemplului de la <https://css-tricks.com/full-page-background-video-styles/>

Bonus 1:

(0.5) galeria animată ([cerință individuală](#))

Bonus 2:

Se poate considera ca bonus să faceți mai multe efecte css care ar depăși cele 0.25 puncte.

Bonus 3:

(0.05) Fișierele salvate în backup (în funcția compileazaScss, în urma compilării scss -> css) să aibă în nume o informație de timp (de exemplu, în loc de a.css să fie fișierul a_timestamp.css, de exemplu a_1681124489791.css) pentru a putea salva mai multe versiuni ale aceluiși fișier.

Bonus 4:

(0.025) În cadrul codului dat la curs, pentru compilarea fișierelor scss, aceasta funcționează corect doar dacă în numele fișierelor nu există puncte (de exemplu nu ar merge pentru stil.frumos.css). Corectați codul astfel încât să funcționeze și pentru acest tip de fișiere.

Bonus 5:

Se va implementa o funcție de verificare a datelor din JSON-ul cu imagini, care va fi apelată la pornirea serverului. Funcția va afișa mesaje de eroare (cu text clar, detaliat și relevant care să explice problema, pentru a fi remediată) în următoare situații:

- (0.025) Folderul specificat în "cale_galerie" nu există în sistemul de fișiere
- (0.025) Nu există (în sistemul de fișiere) vreunul dintre fișierele imagine specificate în lista de imagini.

Etapă 6	22.05.2026	29.05.2026	
---------	------------	------------	--

Taskuri etapa 6 (baza de date + JavaScript introductiv) - afișarea produselor (punctaj recomandat 1.75p)

(0.05) corectarea linkurilor din meniu; **daca nu s-a facut deja acest lucru.**

(1.2p) afisare + sortare/filtrare/calculare (**cerință individuală**)

(0.3p - fiecare e 0.05) **Stilizare inputuri**. Inputurile de pe pagina de produse se vor stiliza cu ajutorul Bootstrap, folosind customizari facute de voi in sass (vezi etapa 5). Puteti sa mai adaugati variabile in fisierul de customizare, care sa ajute la stilizarea butoanelor si inputurilor.

- Butoanele (de filtrare, sortare, resetare) trebuie stilizate cu ajutorul temei bootstrap pentru care ati schimbat culoarea** (de exemplu primary sau secondary). **Raza si grosimea borderului** trebuie date de voi in fisierul sass de customizare prin variabile (daca nu ati facut asta deja). Butoanele si inputurile **vor fi dimensionate cu ajutorul claselor bootstrap**. Butoanele trebuie sa aiba iconuri (glyphicons) relevante din bootstrap: <https://icons.getbootstrap.com/>. Pe ecran mic se vor afisa doar iconurile fara text (realizati acest task in mod cat mai eficient - scris cat mai putin)
- Inputul de tip textarea va avea un **floating label** (bootstrap). In cazul validarii esuate a valorii din textarea (vezi cerinta cu validarea din etapa 5), floating label-ul va fi de tip *is-invalid* (se va seta prin javascript) si se va corecta automat daca valoarea din textarea devine valida.
- Fie inputurile de tip checkbox fie cele radio vor fi stilizate ca **toggle buttons**(din bootstrap). Butoanele deslectate trebuie sa fie desenate in stil outline (adica sa apara doar granita, fara background si textul sa fie colorat), iar la selectare sa apara cu background colorat - vezi clasele *btn-outline*). **Atentie trebuie folosit bootstrap nu css!**
- Asezarea inputurilor** in mod ordonat si aliniat pe coloane in cadrul paginii de produse se va face printr-un **grid bootstrap** (vezi clasele row si col, impreuna cu clasele asociate query-urilor media).
- Butonul pentru schimbarea temei sa fie un switch** din bootstrap (imagineasau iconul cu luna/soarele, ramane in pagina, cu acelasi comportament) **Observatie pt cei care au bonusul cu mai multe teme: puteti inlocui cu alt element de bootstrap.**
- Pentru inputul de tip range**,schimbati cu ajutorul customizarii bootstrap (prin variabile scss) dimensiunea bulinei care gliseaza (sa fie cu 50% mai mare fata de dimensiunea font-size-ului html-ului), schimbati de asemenea culoarea bulinei si culoarea sliderului. Variabilele necesare se gasesc in reference: <https://getbootstrap.com/docs/5.0/forms/range/>

(0.2p) **light/dark theme** cu variabile CSS (optional in SASS). Pentru moment va exista un buton în pagină care va face schimbarea. Butonul va fi reprezentat printr-o imagine cu soare (pt light) vs lună (pt dark) - poate fi un icon din fontawesome. Tema aleasă se va memora în localStorage si se va pastra tema la urmatoarea intrare pe pagina si pe restul paginilor site-ului.

Bonus 1- recomandat - este foarte usor de facut (se da 0.1p pentru fiecare tip distinct de input ajungand la maxim 0.8 puncte pentru cele 8 tipuri de input cerute in enunt) daca se genereaza atributele si eventual eticheta(label) **pe baza informatiilor din tabele si in general din baza de date** (precum select-ul facut impreuna la curs) . De exemplu daca avem un range pentru pret, putem pune ca valoare minima (atributul min) pretul minim din cadrul tabelului si ca valoare maxima (atributul max), pretul maxim.

Bonus 2 (0.15p) Utilizatorul să poată alege dintre 3 sau mai multe teme (nu doar light/dark). Ca și în cazul light/dark, tema aleasă se memorează în localStorage. Modul de alegere a temei se poate realiza printr-un select simplu sau radio buttons.

Bonus 3 (0.05p) Dacă nu sunt produse de afișat în urma filtrării, se va afișa un mesaj corespunzător în locul produselor (de exemplu: "nu exista produse conform filtrării curente").

Bonus 4 (onchange; 0.05 pentru fiecare filtru din cele 8 filtre pe care se aplica efectul - deci poate ajunge la maxim 0.4p). Atentie! Punctarea acestui bonus este conditionata de buna functionare a filtrarii si rezolvarea corecta si completa a cerintelor 6 si 7 din taskul cu produsele! Efectul de filtrare sa se intample la onchange (imediat la schimbarea valorii inputului) in loc sa se fac click pe butonul de filtrare.

Bonus 5 (0.5) Introduceți un sistem de paginare, fie in pagina de produse, fie in cea cu lista de utilizatori, fie in pagina de comenzi.

Se va alege un numar fix, K, de elemente de afisat pe pagina. In cazul in care numarul de elemente de afisat este N, cu $N > K$ se vor genera $NRL = \lceil N/K \rceil$ linkuri, unde cu $\lceil \text{numar} \rceil$ s-a notat partea intreaga superioara a numarului dintre parantezele drepte (de exemplu, $\lceil 2.1 \rceil = 3$). Cele NRL linkuri vor fi numerele de la 1 până la NRL. La click pe un astfel de număr (să-l notăm P) se afișează în pagină produsele cu număr de

ordine între (P-1)*K și P*K-1 (considerând că produsele sunt indexate de la 0).

În loc de linkuri către pagini, puteți folosi și butoane sau select-uri.

Paginarea se poate face fie prin javascript client (se trimit toate produsele dar sunt vizibile doar cele din intervalul corect, fie prin cereri către server care returnează produsele cu indicii între limitele date.

Bonus 6 (0.5=0.2+0.1+0.2) Se vor adăuga trei butoane (cu iconuri, fără text, dar care să simbolizeze rolurile lor) pentru fiecare produs afișat. Când se vine cu cursorul pe fiecare buton se afișează un text explicativ cu rolul lor.

- primul buton va avea rolul de a păstra produsul asociat, chiar dacă o nouă filtrare teoretic l-ar șterge (un fel de cerere de a-l păstra pe ecran tot timpul). La click pe acest buton, atât produsul cât și butonul își schimbă stilul. Produsul nu mai dispăre la filtrare. Dacă se face din nou click pe buton, se dezactivează opțiunea, butonul și produsul revin la stilul inițial și produsul va putea fi din nou ascuns prin filtrare.

- al doilea buton va avea rolul de a șterge temporar produsul asociat, din afișarea curentă (utilizatorul e sigur că nu dorește acel produs).

Acesta poate să reapară la o nouă filtrare, dacă se potrivesc caracteristicile lui, sau o nouă sortare/resetare..

- al treilea buton va avea rolul de a șterge produsul din listă **pe timpul sesiunii curente**. Pentru acel tab în care e deschisă pagina, nu se mai afișează produsul în urma filtrărilor/sortărilor/resetărilor. Această proprietate se păstrează și la refresh sau la plecarea + revenirea în pagină, **dar doar în cadrul tab-ului respectiv**. La deschiderea paginii în alt tab, produsul apare normal. După închiderea tabului pe care s-a făcut setarea, se pierde compet setarea. (Hint: sessionStorage)

Bonus 7 (0.15p) Pentru inputurile de tip text sau elementele de tip textarea care impun căutarea unui cuvânt în cadrul numelui, descrierii sau a altor informații de tip text, simbolurile cu diacritice să fie văzute echivalent cu simburile fără diacritice. De exemplu, dacă în inputul pentru nume căutăm textul "briose" să primim produsul "brioșe". O idee de rezolvare poate fi prelucrearea textului înainte de comparație, prin înlocuirea simbolurilor cu diacritice.

Bonus 8 (0.35p) Utilizatorul să poată alege cele două chei de sortare dorite dintr-un set cu minim 3 valori. Pentru fiecare cheie de sortare va fi un select (sau un alt tip de input care oferă o listă de valori din care se poate alege). Utilizatorul va alege din liste cheile de sortare și apoi va alege dacă vrea sortare crescătoare sau descrescătoare. De exemplu, poate sorta după nume și preț, apoi alege după preț și categorie etc.

Bonus 9 (0.5p) fiecare produs să aibă imagini multiple. Pe pagina proprie produsului se va afișa imaginea principală, iar cu ajutorul unor butoane (sau cu un carusel bootstrap) se va putea trece la imaginea următoare, schimbând imaginea curent afișată. Idee de rezolvare: în baza de date se memorează folderul în care se găsesc toate imaginile produsului (poate să nu fie neapărat folderul, dar din datele produsului programul să poată deduce unde sunt imaginile corespunzătoare lui).

Bonus 10a (0.5p=0.3 filtrare+0.2 sortare după 2 chei crescător/descrescător) **Acest bonus se punctează doar dacă există deja filtrarea și sortarea în JavaScript client.** Să se realizeze sortarea și filtrarea produselor la nivel de server, punând inputurile într-un formular. La click pe butonul de submit, se vor trimite datele din formular către server și serverul va întoarce listă de produse dorită. Pentru această cerință se poate face o pagină separată sau se poate încadra armonios în pagina de produse deja existentă.

Bonus 10b (+0.1p) Punctele pe acest bonus se adună la cele date pe rezolvarea cerinței de mai sus, dacă trimiterea datelor e realizată cu fetch() în loc de formular.

Bonus 11 (0.4p) Pentru fiecare produs, când se da click pe containerul său să apară pe pagina de produse un modal box cu datele produsului (în loc să folosim pagina separată dedicată produsului, să apară direct pe pagina), modal-ul să aibă un buton de ieșire din el sau să se închidă când apăsăm în afara lui. Stilizarea modal-ului se va face cu culori din schema cromatică a site-ului.

Bonus 12 (Total: 1.2p) Se va implementa un **sistem de oferte** astfel:

- (0.5) Se consideră un interval de timp T la care se generează o nouă ofertă (care înlocuiește oferta anterioară). De exemplu, intervalul poate fi o zi (sau, pentru prezentarea bonusului, 1-2 minute). Va exista un fișier json, numit *oferte.json*, care va avea un obiect principal cu proprietatea "oferte", având o valoare de tip vector de obiecte. Obiectele din vector vor avea proprietățile "categorie", "data-incepere", "data-finalizare" și "reducere". La fiecare interval de timp T se va alege aleator o categorie (valorile posibile pentru categorie se vor prelua prin program din baza de date) și o valoare posibilă de reducere dintre valorile: 5,10,15,20,25,30,35,40,45,50. În JSON la începutul vectorului se va introduce noua ofertă, setând și data de începere și de finalizare (va începe din momentul generării și se va finaliza după intervalul T de timp). Nu se vor genera două oferte consecutive pentru aceeași categorie.
- (0.1) Pe prima pagina se va afișa oferta preluată din fișierul JSON (ar trebui să fie mereu prima din vector) sub formă de anunț pentru utilizatori (alegeți voi modul de afișare).
- (0.3) Oferta va avea și un temporizator care va afișa câte ore, minute și secunde mai sunt până expiră (temporizatorul se va actualiza la fiecare secundă). Ultimele 10 secunde vor fi marcate în mod diferit (de exemplu temporizatorul își schimbă culoarea sau generează un efect sonor). Când oferta expiră, aceasta dispăre și e înlocuită de noua ofertă generată.
- (0.2) De asemenea, în pagina de produse, se va afișa pentru produsele din categoria cu ofertă prețul vechi tăiat și alături de el, prețul nou redus cu procentajul din JSON.
- (0.1) Se va considera un interval de timp T2, după care nu ne mai interesează ofertele vechi, prin urmare, ofertele care au expirat în urmă cu mai multe minute decât T2 vor fi șterse din JSON.

Bonus 13 (0.15p) Se consideră un interval de timp T după care fișierele din folderul de backup nu mai sunt de interes, prin urmare vor fi șterse. În mod repetitiv, cu ajutorul unui apel setInterval se va verifica dacă există în folderul backup fișiere mai vechi decât numărul de minute din T, caz în care vor fi șterse.

Bonus 14 (0.3p) Cel mai ieftin produs. În pagina de produse se va marca cel mai ieftin produs din **fiecare** categorie. Modul de marcare îl alegeți voi însă trebuie să fie clar pentru utilizator (de exemplu să scrieți că este cel mai ieftin și să marcați cumva acel mesaj ca să atragă atenția)

Bonus 15 (0.05) În pagina cu produsele se va scrie și **numărul total de produse afișate** (acest număr se va modifica în urma filtrărilor).

Bonus 16 (0.3) Produse similare. Pe pagina descriptivă a unui produs unic se va crea o secțiune de produse similare. Produsele vor fi

selectate prin program din baza de date după un criteriu ales de voi (de exemplu au o specificație comună sau sunt din aceeași categorie). Produsele similare vor avea un afișaj sumar (nume, imagine, eventual câteva caracteristici mai importante). La click pe numele sau imaginea unui produs similar, utilizatorul va fi dus pe pagina celui produs.

Bonus 17 (total 1p) Seturi de produse. Se va realiza astfel:

- a. (0.2) Se va crea un tabel numit "seturi" care va avea coloanele: id, nume set, descriere set. Se va mai crea un tabel numit asociere_set care va avea coloanele id, id_set, id_produs. Tabelele se vor popula astfel încât să existe minim 5 seturi de produse. Un set trebuie să cuprindă minim două produse.
- b. (0.3) În meniul de produse va exista și opțiunea seturi, care va duce spre o pagină specială unde sunt afișate aceste seturi. Modul de afișare va fi ales de voi, dar utilizatorul va avea posibilitatea de a accesa pagina proprie a oricărui produs din set, făcând click pe nume/imagine.
- c. (0.2) Afișarea unui set va cuprinde și prețul acestuia care e calculat prin program astfel: se face suma prețurilor produselor din set și se aplică o reducere de $\min(5, n) \cdot 5\%$, unde n e numărul de produse din set.
- d. (0.3) În pagina proprie a fiecărui produs se vor afișa toate seturile din care acesta face parte, împreună cu prețurile lor. Utilizatorul va avea posibilitatea de a accesa pagina proprie a oricărui produs din set, făcând click pe nume/imagine.

Bonus 18. (0.2) Produse noi. Se va considera un interval T de timp (de exemplu $T = 1$ an - dar pentru prezentare îl veți alege mai scurt) în care un produs e considerat nou pe site; deci produsul e nou, dacă diferența între momentul curent și momentul adăugării este mai mică sau egală cu T . Produsele noi vor avea un marcaj (de exemplu, cuvântul "NOU", stilizat să atragă atenția, pus în containerul lor, pe pagina de produse). De asemenea, într-o secțiune specială, pe prima pagină se vor lista în ordine invers cronologică adăugării cele mai noi produse (de exemplu, pentru a le face reclamă). Produsele vor fi listate cu un link asociat, astfel încât utilizatorul să poată accesa direct pagina produsului. Marcajul de produs nou va fi afișat și pe pagina proprie produsului.

Bonus 19. (0.2) Orarul să poată fi vizualizat pe orice pagină a site-ului (fără a schimba pagina). De exemplu la click pe butonul/linkul de orar să apară un container pe pagină care apoi poate fi închis, dar poate și să dispară singur după un timp, în care este afișat tabelul cu orarul de funcționare. Ziua curentă va fi marcată diferit în tabel (de exemplu cu o culoare de fundal diferită), de asemenea sub tabelul cu orarul se va afișa dacă acum firma e deschisă ori închisă în funcție de **data și ora curentă**. Pentru orar, ca să fie vizualizat ușor pe orice pagină se va face un fragment ejs separat. Orarul va avea toate zilele săptămânii scrise, și intervale orare pentru fiecare, astfel încât macar la anumite ore din anumite zile, firma să fie închisă (studentul va modifica la prezentare tabelul cu orarul pentru a arata cum se schimbă afișajul pentru "închis/deschis").

Bonus 20. (total= 1.5p; e scris în dreptul fiecărui subpunct câte puncte are alocat din total) Se va implementa posibilitatea de a compara două produse (puteți compara toate specificațiile sau alegeți un subset) astfel:

- (0.1p) Pe pagina fiecărui produs, dar și în tabelul de produse va exista un buton cu textul "compară".
- (0.2p) La click pe buton, pentru primul produs, se va crea un container cu id-ul "container-comparare" cu poziție fixă în care va fi afișat numele primului produs de comparat.
- (0.2p) Din momentul în care apare containerul "container-comparare", se va vedea pe orice pagină de produs (care descrie un produs anume) cât și pe pagina cu toate produsele. Containerul nu se va vedea pe paginile care nu afișează produse. Containerul va rămâne afișat în urma acțiunii de reîncărcare a paginii, dar și de închidere și reintrare pe pagină.
- (0.1) Totuși dacă timp de o zi nu s-a mai făcut click pe vreun buton de comparare, containerul va fi ascuns automat.
- (0.3) Va exista posibilitatea de a șterge oricare dintre produsele de comparat (de exemplu, cu un buton în dreptul numelui). Dacă erau 2 produse și s-a șters unul, butoanele cu textul "compară" redevin active. Dacă se șterg ambele produse dispare containerul cu id-ul "container-comparare".
- La click pe butonul "compară" când există deja un produs de comparat se vor întâmpla următoarele:
 - (0.2) Toate butoanele cu textul "compară" se dezactivează. Când se vine cu cursorul pe un astfel de buton dezactivat apare în dreptul cursorului un mesaj informativ cu textul "ștergeți un produs din lista de comparare".
 - (0.1) Numele celui de-al doilea produs va fi afișat în containerul "container-comparare" alături de primul nume. Lângă cele două nume va fi un buton cu textul "afișează".
 - (0.3) La click pe butonul "afișează" se va deschide o nouă fereastră în care se afișează în paralel caracteristicile celor două produse (câte un rand de tabel pentru fiecare caracteristică și câte o coloană pentru fiecare produs).

Etapă 7 (bootstrap avansat, cookie-uri, pregătirea sistemului de utilizatori) (punctaj recomandat 2.4p)

25.05.2026

02.06.2026

(0.3p) Task bootstrap_js (cerință individuală)

(0.3p) Animatie-banner (cerință individuală) - (obiectivele din cerinta: animatie css+cookies). Pentru banner veți folosi un element de forma :
<p id="banner">Acesta este un proiect scolar. Acceptati cookie-urile de pe site? <button id="ok_cookies">Ok</button></p>

Cadru pentru sistemul de utilizatori:

(0.5p) Clasa AccesBD (se rezolva la curs/laborator - punctele se dau de fapt la prezentare pe explicarea functiilor implementate împreună):

Se va implementa într-un fișier javascript dedicat o clasă pentru accesarea bazei de date; să o numim, de exemplu, AccesBD. Această clasă va urma **design pattern-ul Singleton** și va avea următoarele metode și proprietăți:

- o proprietate statică numită instanța, care va conține unica instanță a clasei. Inițial are valoarea null.
- un constructor care va arunca o eroare dacă deja a fost instanțiată clasa.
- una sau mai multe metode de inițializare a bazei de date prin care se oferă datele de autentificare (utilizator, parola, baza de date, port). Aceste metode vor salva în proprietatea client obiectul corespunzător conexiunii prin care se realizează cererile către baza de date. Se va scrie și un getter pentru client.
- o metodă getInstance() care creează o instanță, dacă nu a fost deja creată, și o atribuie variabilei statice instanța. În această metodă se va inițializa și conexiunea la baza de date. Metoda va returna o referință către instanță.
- metodele de accesare și modificare a datelor. Fiecare dintre ele va primi ca parametru un obiect cu proprietățile descrise mai jos (care vor fi folosite pentru crearea query-ului corespunzător), dar și o funcție callback cu 2 parametri (err - setat dacă există o eroare, rez- setat dacă există un rezultat) prin care se va procesa rezultatul query-ului
 - select(obiect, callback). Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile selectate, un vector cu condițiile de selectare (de exemplu un vector de forma ["pret>50", "nume like 'a%'"]).
 - selectAsync(obiect). O metodă asincronă de selectare a datelor din tabel. Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile selectate, un vector cu condițiile de selectare (de exemplu un vector de forma ["pret>50", "nume like 'a%'"]). Metoda va returna rezultatul query-ului.
 - update(obiect, callback). Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile de setat și un vector cu valorile corespunzând câmpurilor (de lungime egală cu cea a vectorului de câmpuri), un vector cu condițiile de selectare a rândurilor care să fie actualizate.
 - insert(obiect, callback). Va insera o înregistrare în tabel. Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu câmpurile tabelului și un vector cu valorile corespunzătoare acelor câmpuri (alternativ se poate da ca parametru un obiect în care proprietățile sunt câmpurile și valorile proprietăților să fie valorile de inserat).
 - delete(obiect, callback). Parametrul obiect va avea ca proprietăți numele tabelului, un vector cu condițiile de selectare a rândurilor care vor fi șterse.

Fișierul javascript va exporta această clasă.

(0.05p) Drepturi (se rezolva parțial la curs/laborator - punctele se dau de fapt la prezentare pe explicarea functiilor implementate împreună):

Realizați un fișier numit drepturi.js în care definiți un obiect (numit **Drepturi**) cu drepturile posibile. Numele proprietăților vor fi identificatori pentru drepturi, iar valorile lor vor fi obiecte de tip Symbol. Trebuie să aveți în fișier minim 7 drepturi (**trebuie să adăugați voi restul**). Aceste drepturi se referă la vizualizare, modificare, ștergere sau adăugare de date. Veți lega aceste drepturi de restul de cerințe.

Fișierul javascript va exporta obiectul **Drepturi**.

(0.25p) Clasa Roluri (se rezolva la curs/laborator - punctele se dau de fapt la prezentare pe explicarea functiilor implementate împreună):

Realizați un fișier numit roluri.js în care definiți o clasă de bază Rol, cu subclasele RolClient (pentru clientul logat), RolAdmin (pentru administratorul site-ului care are absolut toate drepturile), RolModerator (pentru moderatorul site-ului care are toate drepturile legat de utilizatori dar nu poate cumpăra de pe site și nici nu poate vizualiza/modifica/adăuga/șterge produse). Clasa Rol va avea proprietatea publică **cod** în care se va găsi stringul corespunzător rolului, așa cum apare în tabel (de exemplu "admin" pentru rolul de administrator, sau "comun" pentru rolul de client).

Toate clasele de tip Rol vor avea un getter care returnează lista lor de drepturi (simboluri din clasa Drepturi, din drepturi.js).

Clasa Rol are metoda areDreptul(drept) care verifică în lista de drepturi posibile, dacă există dreptul dat ca parametru și returnează true sau false.

(design pattern:factory). Se va implementa o clasă RolFactory cu metoda statică creeazaRol(tip) care primește codul unui Rol și returnează obiectul Rol corespunzător.

Fișierul javascript va exporta clasele Rol și RolFactory.

(0.7p) Clasa Utilizator (se rezolva la curs/laborator - punctele se dau de fapt la prezentare pe explicarea functiilor implementate împreună):

Se va implementa într-un fișier javascript dedicat clasa Utilizator. Pentru această clasă se vor defini:

- proprietăți corespunzătoare câmpurilor din tabelul utilizatori (fiecare coloană va avea o proprietate asociată).
- Se va defini un constructor care setează valori pentru aceste proprietăți prin intermediul unui obiect dat ca parametru (ca în exemplul de la curs). Proprietățile obiectului parametru vor corespunde proprietăților clasei și vor fi folosite pentru inițializarea acestora. Constructorul trebuie să poată fi apelat și fără parametri, caz în care unele proprietăți vor avea valori implicite (default)
- Se vor implementa metode de verificare a datelor. De exemplu: o metodă verificaNume() care verifică dacă numele utilizatorului respectă formatul cerut (menționat în altă cerință), o metodă verificaUsername() care verifică stringul care ar trebui să fie username-ul și eventuale alte funcții de verificare necesare pentru validările cerute la alte subpuncte.

- Metode de interacționare cu baza de date:
 1. O metodă modifica() care primește un obiect cu noile date ale utilizatorului și modifică înregistrarea din tabel corespunzătoare lui. Aruncă o eroare dacă utilizatorul nu există.
 2. O metoda salvareUtilizator() care înregistrează utilizatorul. Dacă username-ul mai există, va arunca o eroare cu un text explicativ.
 3. O metodă sterge() care șterge din tabel utilizatorul curent și aruncă o eroare dacă acesta nu există.
 4. O metodă statică sincronă, getUtilizDupaUsername, care caută un utilizator după username (dat ca parametru). Aceasta va mai primi și un obiect cu niște câmpuri setate și o funcție callback care va folosi acel obiect. De exemplu obiectul poate conține parola introdusă de utilizator la login, iar funcția callback verifică dacă parola corespunde username-ului dat.
 5. O metodă statică asincronă, getUtilizDupaUsernameAsync(username) care returnează obiectul de tip utilizator corespunzător username-ului dat ca parametru. Returnează null dacă nu găsește un astfel de utilizator în tabel.
 6. O metodă statică sincronă cauta(obParam,callback) care primește ca prim argument un obiect ale cărui proprietăți au aceleași nume cu cele ale instanțelor clasei Utilizator. Unele proprietăți din argumentul obiect pot lipsi (au valoare nedefinită), caz în care se va căuta doar după proprietățile inițializate (de exemplu, dacă obiectul e {username: "ionel"} va căuta doar după username. Metoda primește și un callback care are ca argumente: err (care va conține un mesaj de eroare dacă a dat eroare query-ul) și listaUtiliz (un vector cu obiecte de tip utilizator; poate fi și vid dacă nu s-a găsit niciun utilizator cu caracteristicile cerute). Callbackul poate fi folosit pentru procesarea utilizatorilor găsiți
 7. O metodă asincronă cautaAsync(obParam) care primește ca argument un obiect ale cărui proprietăți au aceleași nume cu cele ale instanțelor clasei Utilizator (ca la subpunctul anterior). Metoda returnează o listă de obiecte de tip utilizator corespunzătoare caracteristicilor din obParam. Lista poate fi și vidă, dacă nu s-au găsit utilizatori care să corespundă condițiilor.
 8. O metodă areDreptul(drept) care primește o valoare din drepturi.js și, în funcție de rolul utilizatorului, returnează true, dacă are acel drept și false dacă nu.
 9. O metodă trimiteMail(subiect, mesajText, mesajHtml, atasamente=[]) care trimite un e-mail utilizatorului, cu subiectul și textul dat (atât ca *plain text* cât și ca HTML și o listă de atașamente

Fișierul javascript va exporta clasa Utilizator.

(0.3p) Veți adăuga **JSDoc** (în care se explică scopul funcției, tipul parametrilor și ce returnează) pentru toate clasele și funcțiile din modulele proprii. Se vor adăuga și pentru funcțiile (care nu sunt callback) definite în programul principal. Se acceptă și în TypeScript.

Bonusuri:

Bonus 1. (0.3p) Adăugați în funcțiile select, update și delete, posibilitatea de a avea operatorul logic "or" între condiții. De exemplu, să putem avea și un query de forma "select campuri from tabel where camp1=val1 or camp2=val2 and camp3=val3". Indicație: se poate face cu un vector de vectori de condiții [["a=10", "b=20"], ["c=30", "d=40", "e=50"]] va avea aplicat "and" pe vectorii interni și "or" pe elementele vectorului mare rezultând într-o condiție where de forma "where a=10 and b=20 or c=30 and d=40 and e=50"

Bonus 2. (0.4p) Folositi un ORM precum **Sequelize** (<https://sequelize.org/>) in locul query builder-ului scris impreuna la laborator. **Atentie! Pentru ca acest bonus sa fie punctat este obligatoriu sa aveti un pattern de tip singleton implementat in alta parte in cadrul proiectului!**

Bonus 3. (0.4p) Filtre persistente. Se va afișa în pagina de produse și un checkbox cu eticheta "salveaza filtrare". Dacă acesta se bifează, ultima filtrare făcută cu checkboxul bifat va fi salvată în localStorage (sau cookie) și la reintrarea pe pagină filtrele vor setate pe valorile din ultima filtrare și se vor afișa produsele corespunzătoare lor. La click pe butonul de resetare a filtrării, se debifează și checkboxul de salvare a filtrării și se șterge din localStorage sau din cookie filtrarea anterioară. Indicație: salvăm în localStorage/cookie valorile filtrelor. La încărcarea paginii setăm în inputuri acele valori și reapelăm funcția de filtrare, astfel încât să vedem rezultatele filtrării anterioare

1.			

•	